

# Rust 程序设计课程项目报告

## Mini-Lisp 解释器实现

成员：周静怡 2400013056

王嘉怡 2400013107

Rust 程序设计 (春季, 2026)

北京大学

2026 年 6 月 29 日

# 目录

|                               |          |
|-------------------------------|----------|
| <b>MiniLisp-RS 项目</b>         | <b>2</b> |
| <b>1. 项目概述</b>                | <b>2</b> |
| <b>2. 项目文件结构</b>              | <b>2</b> |
| <b>3. 使用方法与可用功能</b>           | <b>3</b> |
| 3.1 命令行使用方式 . . . . .         | 3        |
| 3.2 图形界面使用方式 . . . . .        | 3        |
| 3.3 当前支持的语言功能 . . . . .       | 4        |
| 3.4 图形界面的辅助功能 . . . . .       | 4        |
| <b>4. 系统设计与实现细节</b>           | <b>5</b> |
| 4.1 总体设计思路 . . . . .          | 5        |
| 4.2 解释器核心模块实现 . . . . .       | 5        |
| <b>5. Rust 使用特点与 C++ 对比分析</b> | <b>6</b> |
| <b>6. 结果展示</b>                | <b>7</b> |
| <b>7. 总结</b>                  | <b>8</b> |
| 7.1 项目亮点与不足 . . . . .         | 8        |
| 7.2 分工说明 . . . . .            | 9        |

# MiniLisp-RS 项目

## 1. 项目概述

Mini-Lisp 是对 Lisp 语言的一种简化实现形式，保留了 S 表达式、符号求值、函数式调用、闭包和列表处理等核心特征。本项目以 Rust 为实现语言，完成了一个轻量级 Mini-Lisp 解释器，并在命令行解释执行的基础上扩展了图形界面。

我们曾在软件设计实践课程中，使用 C++ 实现了一个类似的 Mini-Lisp 解释器，在当时体验到了大量的面向对象编程的思想。因此，我们产生了用 Rust 实现一个新版本的想法，期望借此更深刻地体会 Rust 语言的不同特性。

故而，本项目的主要目标包括三个方面：第一，实现一个能够正确解析并求值 Mini-Lisp 表达式的解释器，支持基本算术、条件判断、变量定义、匿名函数、列表操作和若干内置过程。第二，在此前使用 C++ 实现类似解释器的经验基础上，尝试使用 Rust 重新设计该项目，从而体会 Rust 在所有权、模式匹配、错误处理和模块化组织方面的特点。第三，我们希望在解释器核心之外提供更友好的使用方式，即完成解释器的桌面图形界面。

从最终结果来看，项目已经形成了较完整的系统：既可以通过 REPL 逐行输入 Lisp 表达式，也可以直接执行 .scm 文件；图形界面支持输入区、输出区、按钮控制、语法高亮以及括号匹配提示等功能，使得 Mini-Lisp 的演示和调试更直观。

本项目下载方式和全部代码可见 <https://github.com/Janezzzhou/MiniLisp-RS>

## 2. 项目文件结构

项目的核心语言逻辑位于 `src/` 目录下，图形界面入口位于 `src/bin/` 下。其主要文件结构如下：

`src/`

|                           |                     |
|---------------------------|---------------------|
| <code>lib.rs</code>       | 对外库入口，提供统一执行接口      |
| <code>main.rs</code>      | 命令行入口，负责 REPL 与文件执行 |
| <code>token.rs</code>     | Token 定义            |
| <code>tokenizer.rs</code> | 词法分析器               |
| <code>parser.rs</code>    | 语法分析器               |

|                               |                   |
|-------------------------------|-------------------|
| <code>value.rs</code>         | 运行时值的表示           |
| <code>eval_env.rs</code>      | 求值环境与求值器          |
| <code>form.rs</code>          | 特殊形式实现            |
| <code>builtins.rs</code>      | 内置过程实现            |
| <code>output.rs</code>        | 输出抽象              |
| <code>error.rs</code>         | 错误类型定义和错误打印       |
| <br><code>src/bin/</code>     |                   |
| <code>gui.rs</code>           | GUI 程序入口          |
| <br><code>src/bin/gui/</code> |                   |
| <code>theme.rs</code>         | GUI 主题、颜色、字号、尺寸配置 |
| <code>editor.rs</code>        | 编辑器高亮与装饰逻辑        |

## 3. 使用方法与可用功能

### 3.1 命令行使用方式

项目支持以命令行形式启动交互式 REPL:

```
cargo run --bin mini_lisp_rs
```

按 `Ctrl+Z` 退出。

也支持直接执行一个 Lisp 源文件:

```
cargo run --bin mini_lisp_rs sort_test1.scm
```

(`sort_test1.scm` 是我们提供的排序算法示例文件)

在命令行模式下, 用户可以逐条输入表达式并立即查看求值结果, 这对于调试解释器行为、验证内置函数和测试递归过程都很方便。

### 3.2 图形界面使用方式

除了命令行模式外, 项目还实现了一个独立的桌面图形界面:

```
cargo run --release --bin gui
```

构建完成后，也可以直接点击运行生成的可执行文件，不再依赖 PowerShell：

```
target/release/gui.exe
```

这一点提升了项目的可用性和演示效果。

### 3.3 当前支持的语言功能

目前解释器已经支持以下功能：

- 数值与布尔值的表示和运算
- 符号求值与变量绑定
- `define`、`lambda`、`if`、`begin`、`cond`、`let` 等特殊形式
- 列表、序对以及基础列表处理过程
- `car`、`cdr`、`cons`、`append` 等基础内置过程
- `map`、`filter`、`reduce` 等高阶函数
- 文件输入与批量执行

### 3.4 图形界面的辅助功能

在图形界面部分，项目实现了以下提升体验的编辑器功能：

- 输入框和输出框左右分栏显示，便于对照查看
- 一键执行 (Run)、清空输出 (Clear Output)、重置环境 (Reset Env)
- 查看示例代码和已实现的语言功能（包括特殊形式 (Special Forms) 和内置过程 (Built-in Procedures)）的按钮 (Reference)
- 输入文本的自动语法着色
- 括号匹配提示

## 4. 系统设计与实现细节

### 4.1 总体设计思路

系统整体采用分层设计。最底层是词法分析与语法分析，中间层是运行时值表示、求值环境和表达式求值逻辑，最上层是命令行与图形界面两种输入输出前端。用户输入一段 Lisp 代码后，系统首先将源码拆分为 token，再将 token 解析为内部表达式结构，最后在求值环境中执行并返回结果。

### 4.2 解释器核心模块实现

**词法分析模块** 词法分析器 tokenizer 负责将用户输入的字符串拆分成一系列 token。

例如，对于输入：

```
(+ 1 2)
```

词法分析器会将其转换为：

```
LeftParen, Identifier("+"), Number(1), Number(2), RightParen
```

该模块主要处理括号、数字、字符串、布尔值和标识符等基本词法单元。

**语法分析模块** 语法分析器 parser 负责根据 token 序列构造 Mini-Lisp 表达式。由于 Lisp 的语法结构非常统一，主要由原子表达式和列表表达式组成，因此语法分析的核心是递归处理括号嵌套结构。

例如：

```
(+ 1 (* 2 3))
```

会被解析为一个嵌套表达式，其中外层表达式表示 + 函数调用，内层表达式表示 \* 函数调用。

**运行时值表示** 本项目使用 enum 表示不同类型的运行时值，例如 Numeric、Boolean、Pair 等，并通过 match 对不同情况进行处理。

**求值环境** 求值环境用于保存变量名到值的绑定关系。例如执行：

```
(define x 10)
```

之后，环境中会记录 `x` 对应的值为 10。当之后输入：

```
(+ x 5)
```

解释器会先在环境中查找 `x`，再进行求值。项目中，环境不仅负责保存当前作用域内的变量绑定，还通过父环境指针形成链式作用域结构，从而支持一些函数变量的局部作用域需求。相关逻辑主要由 `eval_env.rs` 管理。

**表达式求值** 求值器是解释器的核心模块。它根据表达式的不同形式执行不同逻辑：

- 如果表达式是数字、布尔值或字符串，则直接返回自身。
- 如果表达式是符号，则在环境中查找其绑定值。
- 如果表达式是列表，则将其视为特殊形式或函数调用。
- 如果表达式不合法，则返回相应错误。

**特殊形式与内置过程** 在 Lisp 中，并不是所有表达式都遵循统一规则，例如 `define`、`if`、`lambda`、`begin`、`cond` 和 `let` 都具有自己的求值语义。因此，我们将这类语法结构单独放入 `form.rs` 中实现，以便与普通函数调用区分开来。

我们在 `builtins.rs` 中集中实现具体内置过程的逻辑，主要包括算术运算、比较运算、列表操作以及若干高阶函数。

## 5. Rust 使用特点与 C++ 对比分析

在项目中，我们也切身体会到了 Rust 与 C++ 编程语言风格的不同。

比如最开始做 C++ 版本时，我们使用大量的面向对象设计模式，将 `Value` 作为抽象基类，再派生出数值、字符串、布尔等不同子类，每个子类都有自己的实现。而在 Rust 版本中，我们使用枚举来表示不同类型的值，每个枚举变体对应一种类型的值，通过 `match` 对不同情况进行处理。对于其他类似面向对象中“共享行为”的需求，在 rust

中也会使用 `struct` 加上 `trait` 的方式实现。在当前的解释器代码编写体验里，Rust 版本似乎是更舒适一些的。

在原本的 C++ 实现中，我们大量使用 `std::shared_ptr` 管理值对象和环境对象，以便在表达式、环境链和闭包之间共享数据。这是更加方便的，但需要程序员自行维护何时访问何时修改的问题。Rust 实现虽然同样使用引用计数，但会把共享和修改区分开。对于运行时值，我们使用 `Rc<Value>` 可共享；而对于求值环境，我们使用 `Rc<RefCell<EvalEnv>`，表示被共同持有的同时还需被修改。但这样做的代码复杂度也大大增加了。

在错误处理时，C++ 版本里若使用异常，错误传播路径有时不那么直观；如果不用异常，又容易变成手动检查返回值，比较麻烦。Rust 里的 `Result<T, E>` 机制要求错误处理直接体现在函数签名中。本项目中的求值也大量采用 `Result<ValuePtr, LispError>` 返回值，使语法错误、运行时错误以及类似退出解释器这样的控制流都能够在类型层面被统一描述。

## 6. 结果展示

本项目已经能够支持 Mini-Lisp 的基本解释执行，并提供命令行与图形界面两种交互模式。

以下为项目运行截图展示：

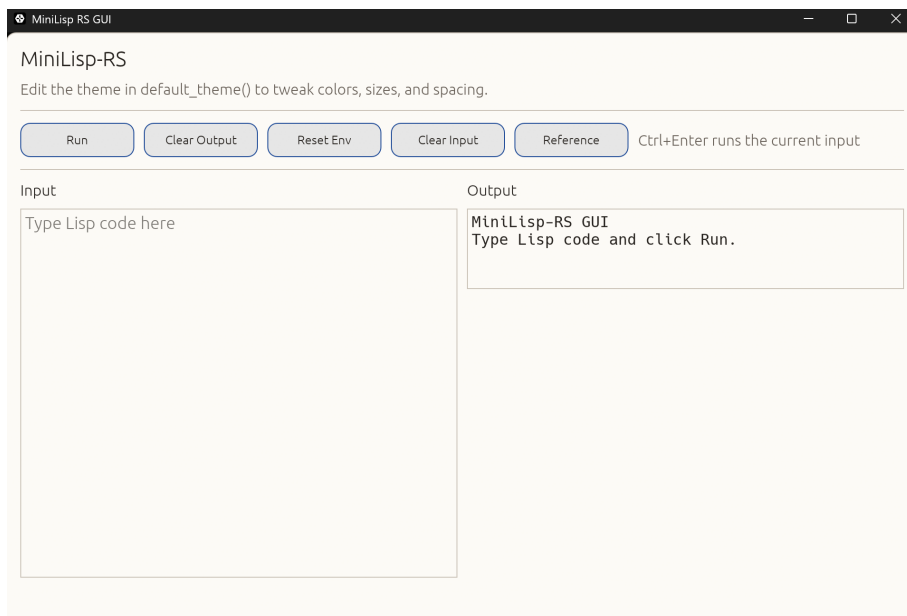


图 1: 初始界面展示

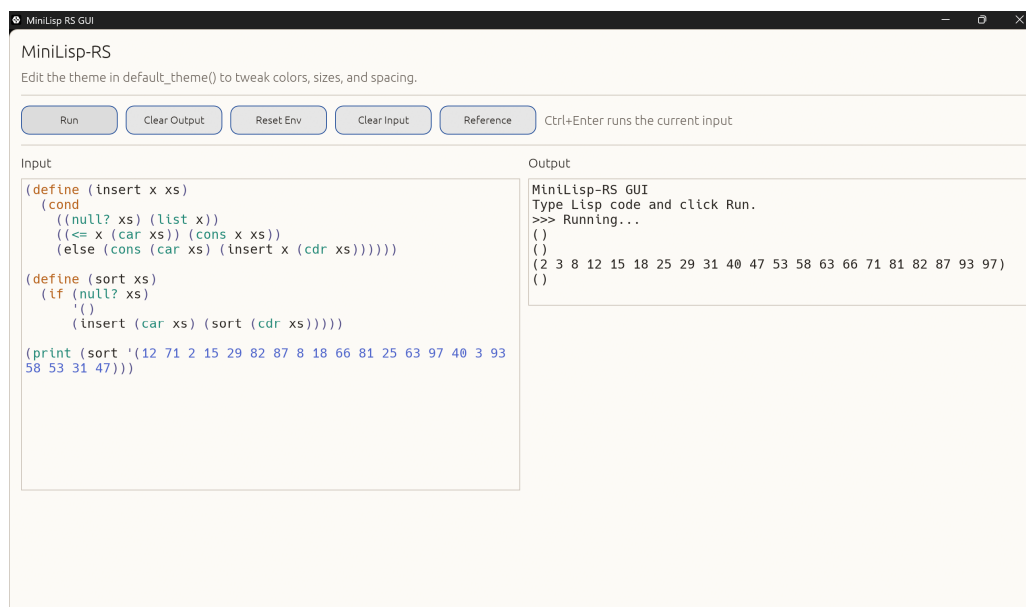


图 2: 运行效果展示

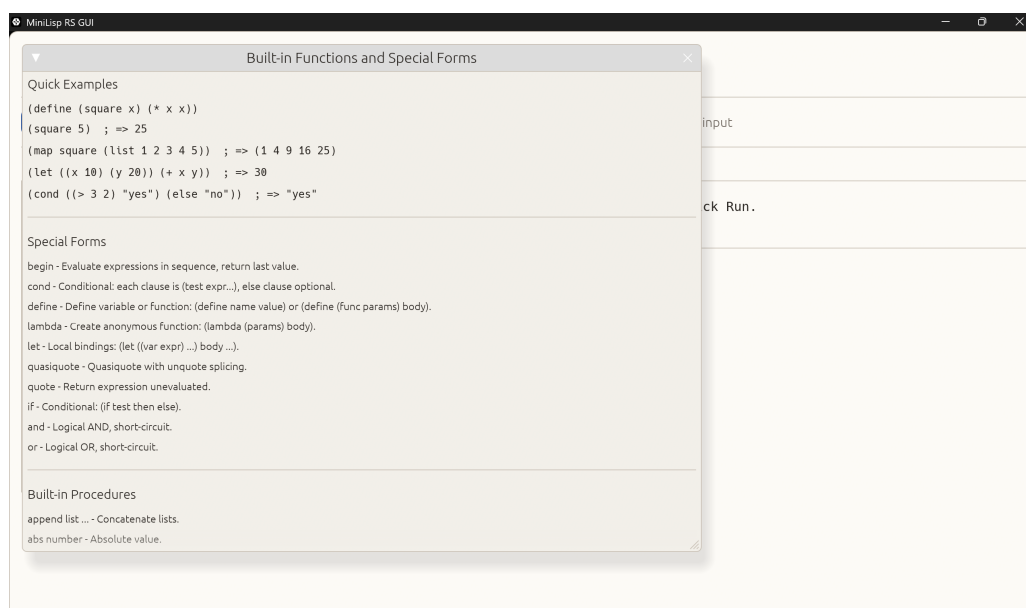


图 3: 教程文档功能展示

## 7. 总结

### 7.1 项目亮点与不足

本项目的主要亮点在于：

- 较完整地实现了一个 Mini-Lisp 解释器，覆盖了解析、求值、闭包、特殊表单和高阶函数等关键内容。
- 解释器核心之外实现了独立桌面 GUI，提供较好的使用体验和可视化结果。
- 在实现过程中体现了 Rust 与 C++ 不同的工程思维方式，为后续语言实现学习提供了比较样本。

当然，项目仍有进一步完善空间：

- 目前 GUI 仍以轻量级编辑器体验为主，尚未加入更复杂的调试辅助能力。
- 解释器支持的语言特性也仍然是 Mini-Lisp 级别，没有涉及宏系统以及更完整的标准库。

## 7.2 分工说明

本次项目的分工如下：

- 周静怡：负责项目初始框架搭建，完成早期词法分析器、语法分析框架的建立；参与后续解释器功能扩展，包括 `lambda` 等部分特殊形式、添加具体内置过程；完善文件输入功能；完成 GUI 界面实现，加入自动着色功能；报告撰写。
- 王嘉怡：负责语法分析器的完善与接口调整，完成解释器求值、变量定义与查找、调用内置过程等功能；进行解释器调试测试与 bug 修复；完成 GUI 功能优化，包括输入清除、示例代码和语言规则查看功能。

本项目下载方式和全部代码可见 <https://github.com/Janezzzhou/MiniLisp-RS>